

I118 TALLER DE PROYECTO II

Entrega 1: Plan de Proyecto

LoRa OTA UpdateGrupo J5Bermúdez, Lautaro Joaquín¹ (01399/0)Majoros, Lorenzo² (03296/1)Seery, Juan Martín^{2*} (03471/9)

13/09/2026

¹lautarojbermudez@gmail.com²{lorenzomajoros,juan.seery}@alu.ing.unlp.edu.ar

*Autor responsable del informe

Índice

Nomenclatura	1
1. Introducción	2
2. Objetivo	2
2.1. Requerimientos	3
2.2. Métricas	4
3. Esquema gráfico del proyecto	5
4. Identificación de partes	6
4.1. Partes de hardware	6
4.2. Requerimientos de alimentación	6
4.3. Partes de software	6
4.3.1. Firmware del nodo	6
4.3.2. Panel de control	7
5. Cronograma de tareas	8
5.1. Primer informe de avance (octubre)	8
5.2. Segundo informe de avance (noviembre)	8
5.3. Entrega final (diciembre)	9
6. Documentación en Video	9
Bibliografía	9

Nomenclatura

OTA	(<i>Over-the-air</i>) Actualización de software distribuida por un enlace inalámbrico
MCU	(<i>Micro-controller unit</i>) Microcontrolador
VM	(<i>Virtual machine</i>) Intérprete de instrucciones
LoRa	(<i>Long Range</i>) Modulación de radio de largo alcance
SF	(<i>Spreading factor</i>) Factor de dispersión
CAD	(<i>Channel activity detection</i>) Detección de actividad en el canal
PWA	(<i>Progressive web app</i>): Aplicación web instalable que funciona sin conexión
RSSI	Indicador de intensidad de señal recibida
SNR	Relación señal a ruido

1. Introducción

Las redes de dispositivos embebidos distribuidos a lo largo de un territorio, como sensores agrícolas, estaciones meteorológicas o nodos de telemetría, comparten un problema práctico: una vez instalados, actualizar el software que ejecutan exige que un operario se movilice hasta cada uno de los nodos. La actualización remota OTA resulta una solución atractiva a ese problema. Típicamente, las actualizaciones OTA se realizan sobre WiFi o red celular, como es el caso de los teléfonos celulares, pero eso limita el alcance y la complejidad de los dispositivos. LoRa, en cambio, ofrece kilómetros de alcance a cambio de tasas de datos de unos pocos kilobits por segundo y paquetes de a lo sumo 255 B.

El problema tiene antecedentes en las redes de sensores inalámbricos, donde protocolos como Deluge [1] y Trickle [2] resolvieron la propagación de código sobre radiofrecuencia de baja potencia mediante disseminación epidémica y reenvío selectivo. Este proyecto toma esas ideas y las lleva a un escenario acotado — una red de microcontroladores ESP32 con antenas LoRa donde se distribuye un *símil firmware*: programas para una VM ligera que cada nodo ejecuta.

En su propuesta más básica, este proyecto busca abarcar dos MCU con módulos LoRa y una micro VM para procesamiento numérico básico en cada uno. Cualquier nodo puede conectarse por USB a una PC y recibir la nueva versión del *firmware* y retransmitirla a su par, además de coordinar entre ambos para ejecutar la versión nueva. El grupo propone además extender el sistema a más de dos nodos, de modo que la retransmisión punto a punto sea el caso particular de un protocolo de inundación en malla. Esa extensión está planteada como objetivo secundario y condicionada a la disponibilidad de hardware adicional por parte de la cátedra.

2. Objetivo

El objetivo general del presente proyecto es diseñar, implementar y evaluar un sistema de actualización OTA de programas ejecutables sobre una red descentralizada de nodos ESP32 con LoRa, en el que cada nodo corre los programas sobre una micro VM propia.

En particular, los **objetivos principales** se desglosan en:

- a) especificar e implementar una micro VM de bytecode para procesamiento numérico entero, con su formato de programa, su herramienta de ensamblado y un intérprete de referencia que permita verificar la correctitud de la ejecución en los nodos;
- b) especificar e implementar un protocolo de distribución OTA sobre LoRa entre dos nodos, con fragmentación, detección y reparación de pérdidas y activación coordinada de la versión nueva con confirmación al origen;
- c) permitir que cualquier nodo actúe como origen de una actualización al conectarlo por puerto serie (USB) a la PC, sin roles fijos;
- d) implementar las herramientas necesarias para analizar métricas de desempeño y correctitud y analizar los resultados;
- e) desarrollar un panel de control para la PC que se conecte por puerto serie (USB) a cualquier nodo para cargar programas, observar el estado de la red y visualizar las métricas.
- f) asegurar el origen de los programas mediante firma digital de la imagen, autenticación de cada paquete para evitar manipulaciones externas;

Además, se define como **objetivos secundarios**:

- a) extender el protocolo a tres o más nodos en malla, con retransmisión multisalto, control de colisiones entre retransmisores y descubrimiento automático de los nodos de la red mediante mensajes periódicos de presencia;
- b) extender las métricas definidas a una red en malla y analizar los resultados;
- c) armar un nodo portable alimentado a batería que permita conectarse a la red desde cualquier punto.

2.1. Requerimientos

Requerimientos funcionales

- RF1) **Ejecución continua.** Cada nodo debe ejecutar de forma permanente el programa que tiene activo, y sus resultados deben poder observarse desde una PC conectada a cualquier nodo de la red.
- RF2) **Carga.** Un operador debe poder cargar una nueva versión de programa desde una PC conectada por puerto serie a cualquier nodo, sin que ese nodo requiera un rol, un firmware o una configuración distintos de los demás.
- RF3) **Distribución.** Toda versión cargada en un nodo debe llegar completa e idéntica a todos los demás nodos de la red, aun cuando el enlace de radio pierda paquetes.
- RF4) **Activación coordinada.** Ningún nodo debe ejecutar la versión nueva antes de que el nodo origen lo ordene y el origen solo debe ordenarlo cuando todos los nodos conocidos dispongan de la versión completa. Hasta ese momento, cada nodo debe seguir ejecutando su versión anterior sin interrupciones.
- RF5) **Confirmación.** El nodo origen debe informar al operador cuándo todos los nodos de la red quedaron ejecutando la versión nueva y, vencido un plazo, cuáles no lo hicieron. El operador debe poder forzar la activación en los nodos que ya disponen de la versión o abortar la actualización.
- RF6) **Convergencia.** Si se inician dos actualizaciones de forma concurrente desde nodos distintos, la red debe converger a una única versión, la más nueva, y nunca deben coexistir dos programas distintos bajo el mismo número de versión. Un conflicto de ese tipo debe ser visible para el operador.
- RF7) **Aislamiento de fallas.** Un programa defectuoso (lazo infinito, división por cero, acceso inválido) no debe impedir que el nodo siga participando de la red ni recibiendo actualizaciones. El error debe quedar registrado y ser visible para el operador.
- RF8) **Verificabilidad.** Debe poder comprobarse desde la PC que el resultado que produce cada nodo es el esperado para el programa y la versión que tiene activos.
- RF9) **Reinicio.** Un nodo que se reinicia debe quedar en un estado conocido, informarlo al resto de la red y volver a la versión vigente de forma automática, sin intervención del operador.
- RF10) **Observabilidad.** Desde una PC conectada a cualquier nodo, el operador debe poder conocer la versión que ejecuta cada nodo, sus últimos resultados, sus errores, el progreso de la actualización en curso y la calidad de los enlaces de radio.
- RF11) **Registro de métricas.** El sistema debe registrar por sí mismo los datos necesarios para calcular las métricas de la [Tabla 1](#) y permitir exportarlos.
- RF12) **Autenticidad.** Un nodo solo debe instalar versiones cuya procedencia de un operador autorizado y cuya integridad pueda verificar. Debe descartar sin efecto alguno cualquier imagen o mensaje que no supere esa verificación. La capacidad de autorizar una versión debe residir en el operador y no en los nodos: un nodo comprometido no debe poder hacer que los demás acepten un programa.
- RF13) **Alcance multisalto (secundario).** Una actualización debe llegar a nodos que están fuera del alcance de radio directo del origen, siempre que exista una cadena de nodos intermedios que los conecte.
- RF14) **Descubrimiento (secundario).** La red debe determinar por sí misma qué nodos la integran, sin una lista configurada de antemano, y reflejar la aparición o la desaparición de un nodo dentro de un plazo acotado.

Requerimientos no funcionales y de tiempo real

- RNF1) **Tiempo.** No hay requerimientos de tiempo real estricto. Sí se exige que la ejecución de un programa nunca demore la atención de la radio más allá de una cota fija, de modo que ningún paquete se pierda por culpa del programa, y que una actualización completa entre dos nodos se complete en un tiempo compatible con una demostración en vivo.
- RNF2) **Autonomía.** El sistema debe funcionar sin conexión a Internet. Una actualización ya iniciada debe completarse aunque la PC se desconecte del nodo origen.
- RNF3) **Uniformidad.** Todos los nodos deben correr el mismo firmware, sin variantes por rol, y el diseño debe admitir al menos cuatro nodos sin cambios en él.

- RNF4) **Entregables.** El firmware se entrega como proyecto de Arduino, según lo requiere la cátedra. El panel de control debe poder ejecutarse sin conexión a Internet en una PC con navegador basado en Chromium.
- RNF5) **Regulación.** El sistema debe operar en la banda de uso libre de 915 a 928 MHz vigente en Argentina [3].

2.2. Métricas

Aparte del funcionamiento intrínseco de lo construido, se realizará especial énfasis en medir el rendimiento logrado. Para ello, se definen distintas métricas que la red podrá medir por sí misma a partir del flujo de eventos que emiten los nodos. Las mismas se listan en la [Tabla 1](#).

Tabla 1. Métricas definidas para el sistema.

Métrica	Definición	Unidad	Método
M1. Tiempo de convergencia	Desde que la PC entrega la imagen al origen hasta que el último nodo confirma la activación	s	Marcas de tiempo del origen; en función del tamaño de imagen y de la cantidad de nodos
M2. Eficiencia de la inundación	Transmisiones totales en la red dividido el mínimo teórico (fragmentos por cantidad de nodos menos uno)	—	Contadores de transmisión de cada nodo reportados en los mensajes de presencia
M3. Pérdida y reparación	Porcentaje de fragmentos faltantes en el primer reporte de cada nodo y cantidad de rondas de reparación	%	Bitmaps de los reportes recibidos por el origen
M4. Latencia de ida y vuelta	Tiempo entre un mensaje dirigido a un nodo y su respuesta	ms	Marcas de tiempo del origen; por cantidad de saltos
M5. Correctitud de la VM	Ticks cuya salida coincide con el intérprete de referencia sobre el total	%	Comparación en el panel entre lo reportado por los nodos y la ejecución de referencia
M6. (<i>secundaria</i>) Calidad de enlace	RSSI y SNR de los mensajes de presencia de cada vecino	dB	Lectura del transceptor en cada recepción
M7. (<i>secundaria</i>) Recursos	Uso de RAM y flash del firmware	bytes	Reporte del compilador y del sistema en ejecución

3. Esquema gráfico del proyecto

La [Figura 1](#) muestra las partes físicas del sistema y su relación con las partes funcionales. Cada nodo es una placa ESP32 con transceptor LoRa, antena y alimentación propia, y corre el mismo firmware con tres componentes: radio y protocolo, micro VM y consola serie. La PC del operador corre el panel de control y se conecta por USB al nodo que en ese momento actúe como origen. Los nodos adicionales, dibujados en línea de trazos, corresponden al objetivo secundario de malla.

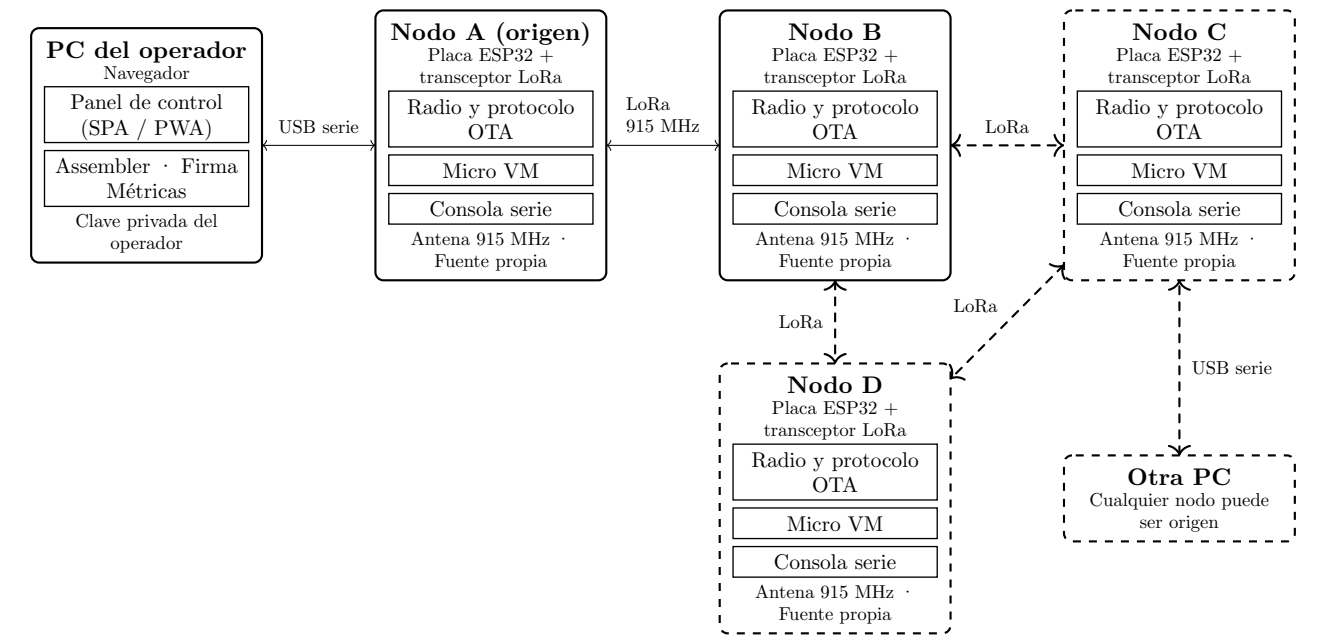


Figura 1. Esquema del sistema.
En trazo continuo el objetivo principal con dos nodos; en trazo discontinuo la extensión a malla.

4. Identificación de partes

4.1. Partes de hardware

Lista de materiales prevista:

- 2 placas [Heltec WiFi LoRa 32 \(V2\)](#), cada una con
 - un *master chip* ESP32,
 - un *transceiver* LoRa SX1276/SX1278,
 - un display OLED de 0,96 pulgadas;
- 2 antenas para 915 MHz con interfaz IPEX1.0;
- 2 cables USB de datos para la conexión serie con la PC;
- (*secundario*): nodos adicionales completos;
- (*secundario*): batería de litio 3,7 V y 1000 mAh.

4.2. Requerimientos de alimentación

Cada nodo tendrá un circuito de alimentación propio y no se alimentará desde la placa de desarrollo ni desde el puerto USB de la PC. Los valores de tensión y corriente de cada parte se detallan en la [Tabla 2](#).

Tabla 2. Requerimientos de alimentación por nodo.

Parte	Tensión	Corriente pico	Corriente típica
Placa ESP32	5 V	240 mA	40 mA
Transceptor LoRa transmitiendo	3.3 V	120 mA	120 mA
Transceptor LoRa recibiendo	3.3 V	12 mA	10 mA
Misc. (OLED, LED, botones, etc.)	3.3 V	30 mA	20 mA
Total por nodo	5 V	390 mA	200 mA
Total del sistema (2 nodos)	5 V	780 mA	400 mA

El módulo cuenta con un conector SH1.25-2 con un sistema de control de baterías de litio.

Esto permite la portabilidad del nodo con baterías de litio de 3.7v y 1000mAh, con un diseño enfocado en el bajo consumo la corriente en reposo es menor a 800uA, por lo que se estima lograr una autonomía de semanas.

4.3. Partes de software

El software se divide en el firmware de los nodos, idéntico en todos, y el panel de control que corre en la PC. Se describen como procesos, entendidos como actividades concurrentes con responsabilidades separadas.

4.3.1. Firmware del nodo

El firmware se desarrolla como proyecto de Arduino para una placa ESP32. Corre en un único bucle principal sin tareas explícitas, con los siguientes procesos:

- Radiofrecuencia y retransmisión.** Recibe cada paquete LoRa, verifica su código de autenticación, lo entrega al proceso correspondiente y lo retransmite (en malla). Debe considerar la propagación aun ante fallas de la deduplicación, en la línea de las soluciones al problema de la tormenta de difusión [4].
- Protocolo OTA.** Máquina de estados que implementa la propagación de firmware, tanto en el rol de origen como en el de receptor, con los plazos y límites de rondas correspondientes. .
- Presencia.** Emite periódicamente mensajes de estado con identificador, versión activa, versión en recepción, salidas de la VM, último error de la VM y contador de transmisiones, además de mantener la tabla de nodos con lo que recibe de los demás. Con dos nodos alimenta la tabla del origen; con más nodos es además el mecanismo de descubrimiento.
- Micro VM.** Ejecuta el programa activo una vez por tick. Es una máquina de pila con enteros de 32 bits con signo, alrededor de 30 instrucciones (carga de constantes, aritmética y lógica, comparación,

saltos, llamadas, variables globales que persisten entre ticks, entradas como identificador de nodo y número de tick, y salida), un tope de instrucciones por tick y detección de condiciones anormales (pila vacía o llena, división por cero, instrucción inválida, salto fuera de rango), que terminan el tick y quedan registradas. La ejecución es determinista: el mismo programa con las mismas entradas produce la misma salida en todos los nodos, lo que hace verificable la correctitud contra un intérprete de referencia.

- e) **Consola serie.** Recibe comandos (cargar imagen, consultar estado, forzar activación, abortar) y emite el flujo de eventos con marca de tiempo del que se obtienen las métricas. El mismo flujo es legible desde el monitor serie del IDE de Arduino, lo que permite operar y demostrar el firmware sin el panel.

Seguridad. La fuente confiable es el operador que carga el programa. Por eso la imagen se firma en la PC con Ed25519 [5] sobre versión, tamaño y hash SHA-256 y los nodos solo tienen la clave pública —ningún nodo puede fabricar una imagen válida. Además, todos los paquetes llevan un HMAC con una clave de red compartida, de modo que un nodo descarta de inmediato tráfico ajeno o alterado sin gastar memoria ni procesamiento. Las claves se compilan en el firmware; su generación y distribución quedan fuera del alcance.

Formato de programa. Una imagen es una cabecera (magic byte, versión de 32 bits, longitud, banderas) seguida del código, con un máximo a definir. La imagen activa no se persiste en memoria no volátil: un nodo reiniciado vuelve al programa de fábrica y pide el firmware a un nodo vecino. La puesta al día de ese nodo es automática. El control de acceso al medio entre retransmisores (espera aleatoria y detección de actividad de canal) es necesario a partir de tres nodos y se define con el hardware en mano.

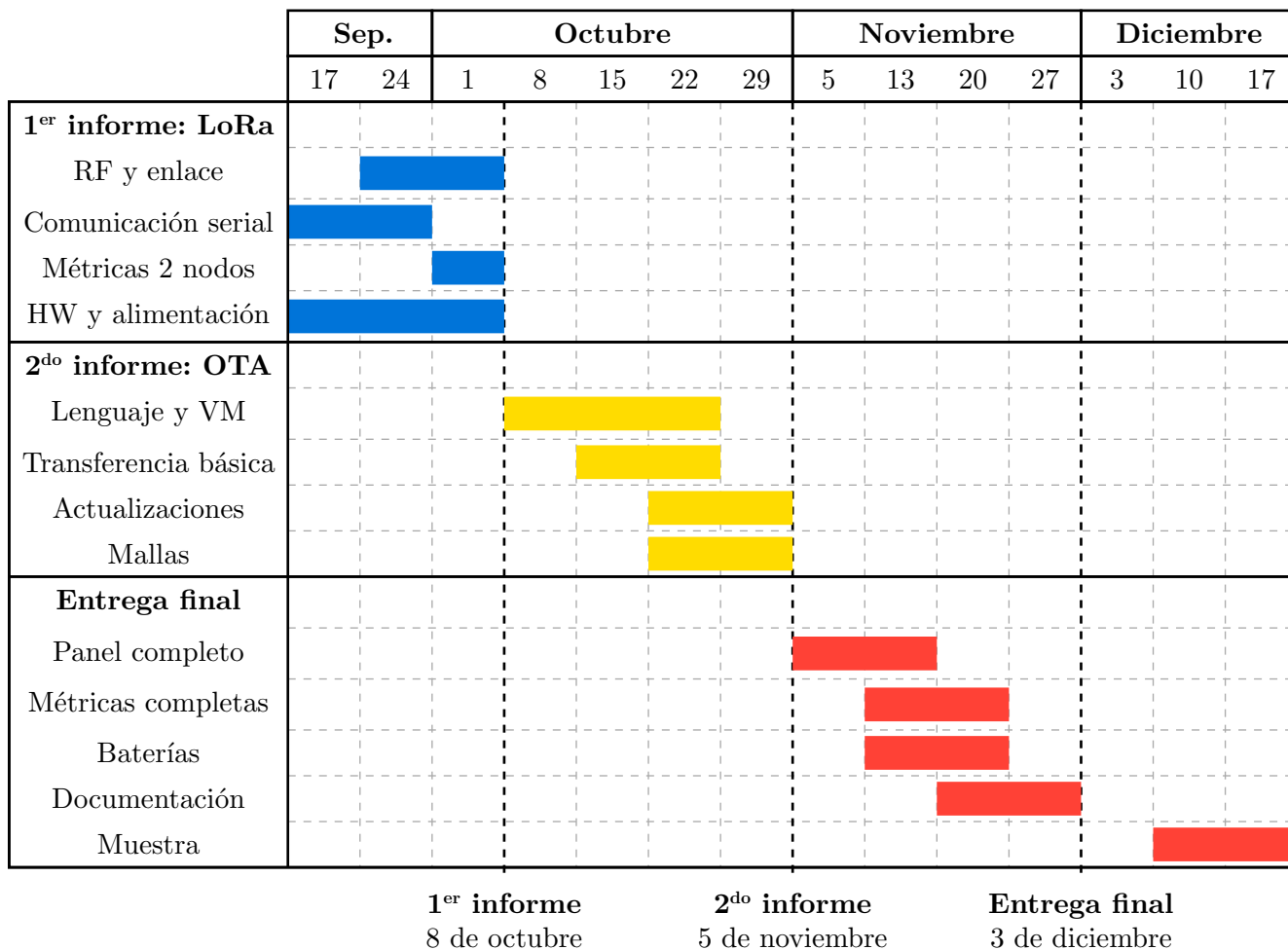
4.3.2. Panel de control

Aplicación web estática hospedada como PWA, instalable para funcionar sin conexión, que se comunica con un nodo por la API Web Serial [6] del navegador. Sus capacidades son:

- a) **Conexión serie.** Abre el puerto elegido por el operador, envía comandos y recibe el flujo de eventos del nodo.
- b) **Ensamblado y firma.** Traduce el programa escrito en un lenguaje ensamblador simple al bytecode de la VM, arma la imagen, la firma con la clave privada que el operador carga desde un archivo y la envía al nodo.
- c) **Intérprete de referencia.** Ejecuta el mismo bytecode en el navegador para calcular las salidas esperadas y compararlas con las reportadas por los nodos (métrica M5). Permite además desarrollar y probar programas antes de disponer del hardware.
- d) **Estado y métricas.** Muestra la tabla de nodos, el progreso de la actualización, la calidad de enlace y los errores de la VM; acumula los eventos, calcula las métricas de la [Tabla 1](#) y las exporta.

Se optó por no servir páginas web desde el ESP32 para no cargar al nodo con un servidor y una interfaz Wi-Fi que no aportan al problema y en cambio servir la interfaz web desde la misma PC. Esto se logra con una *single-page application* (SPA) que el usuario podría descargar de antemano para un uso sin conexión u obteniéndola de un servidor web externo.

5. Cronograma de tareas



5.1. Primer informe de avance (octubre)

- **Radio:** enlace LoRa operativo entre los dos nodos con la cabecera definida, deduplicación y mensajes HELLO con versión y RSSI. Tiempo de aire medido y comparado con el calculado; parámetros de radio fijados.
- **Consola:** flujo de eventos por serie legible desde el monitor del IDE de Arduino. Panel de control mínimo: conexión por Web Serial y visualización del flujo de eventos.
- **Métricas:** las siete definidas por escrito con método de medición; primeras mediciones de tiempo de aire y de tiempo de convergencia con dos nodos.
- **Hardware:** sección de partes y alimentación de este plan completada; alimentación independiente de cada nodo armada y verificada.

5.2. Segundo informe de avance (noviembre)

- **VM:** especificación del conjunto de instrucciones y del formato de imagen; intérprete corriendo en el ESP32; ensamblador e intérprete de referencia en el panel; juego de programas de prueba con salidas coincidentes entre referencia y placa.
- **OTA estático:** un programa cargado por serie en el nodo A llega al nodo B, ambos verifican firma y hash, activan la versión nueva de forma coordinada y la ejecutan.
- **OTA dinámico:** rondas de reparación, plazos, activación forzada y aborto desde el panel, y reversión automática si la versión nueva falla en su primer tick.
- **Malla (*secundario*):** tres o cuatro nodos con retransmisión, espera aleatoria y saltos restantes; descubrimiento de nodos por HELLO; actualización completa con el origen en cualquier nodo.

5.3. Entrega final (diciembre)

- **Panel completo:** carga de programas con firma, tabla de nodos, progreso de la actualización, gráficos de métricas, exportación, instalable como PWA sin conexión.
- **Métricas:** medidas con dos nodos y, de haber hardware, con tres o más; primer análisis de resultados.
- **Portabilidad (*secundario*):** nodo portable a baterías y con cápsula de plástico.

6. Documentación en Video

La presentación del plan de proyecto en video se encuentra en <https://jms.ar/tdp2-e1-video>.

Bibliografía

- [1] J. W. Hui y D. Culler, «The Dynamic Behavior of a Data Dissemination Protocol for Network Programming at Scale», en *Proceedings of the 2nd International Conference on Embedded Networked Sensor Systems (SenSys '04)*, ACM, 2004, pp. 81-94. doi: [10.1145/1031495.1031506](https://doi.org/10.1145/1031495.1031506).
- [2] P. Levis, N. Patel, D. Culler, y S. Shenker, «Trickle: A Self-Regulating Algorithm for Code Propagation and Maintenance in Wireless Sensor Networks», en *Proceedings of the 1st Symposium on Networked Systems Design and Implementation (NSDI '04)*, USENIX, 2004.
- [3] Ente Nacional de Comunicaciones (ENACOM), «Reglamento sobre el uso de la banda de 915 a 928 MHz por sistemas de espectro ensanchado y modulación digital de banda ancha». 2026.
- [4] S.-Y. Ni, Y.-C. Tseng, Y.-S. Chen, y J.-P. Sheu, «The Broadcast Storm Problem in a Mobile Ad Hoc Network», en *Proceedings of the 5th Annual ACM/IEEE International Conference on Mobile Computing and Networking (MobiCom '99)*, ACM, 1999, pp. 151-162.
- [5] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, y B.-Y. Yang, «High-speed high-security signatures», *Journal of Cryptographic Engineering*, vol. 2, n.º 2, pp. 77-89, 2012.
- [6] Web Platform Incubator Community Group, «Web Serial API». [En línea]. Disponible en: <https://wicg.github.io/serial/>